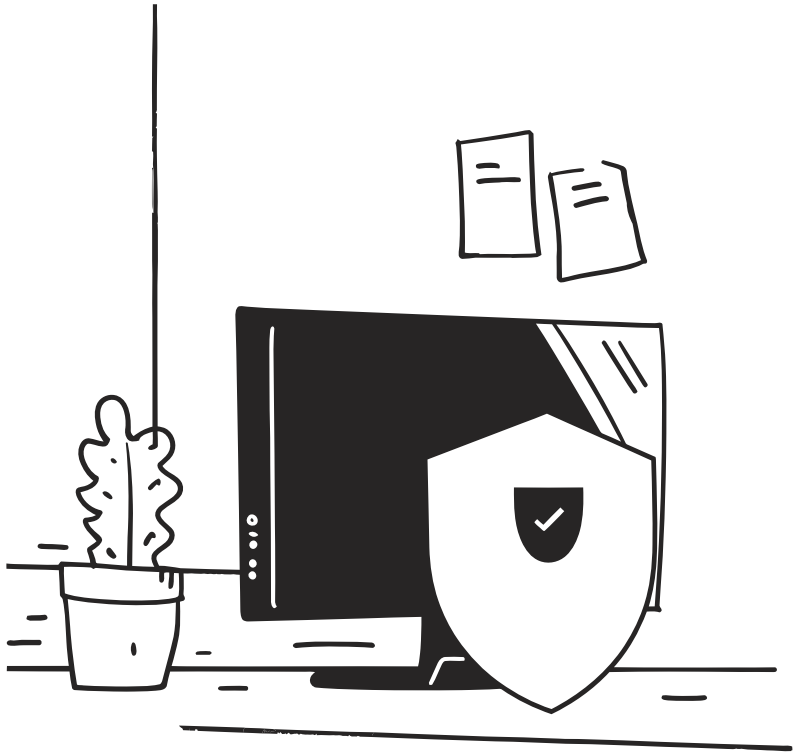
TASS

Prevent Breaches *Before* They're Built



Sreenidhi Nallajerla

The Most Dangerous Sentence in Software Security?

Every security engineer has heard it: **"Nobody told me I couldn't do that."**

A developer or an AI coding assistant quietly adds functionality to your code: an external API call, a new database write path, or network egress in a pull request.

Security doesn't find out until an incident review months later. By then, the code is in production, the architecture is locked in, and fixing it means rewriting half the service.

This isn't a bug anymore. It's an unapproved capability that was never reviewed, never approved, and never caught.

69%

Of organizations in US, Europe
found serious vulnerabilities in code generated by AI

Source: Aikido, State of AI in Security & Development (2026)

Your Security Stack Has a Blind Spot

SAST / DAST Tools

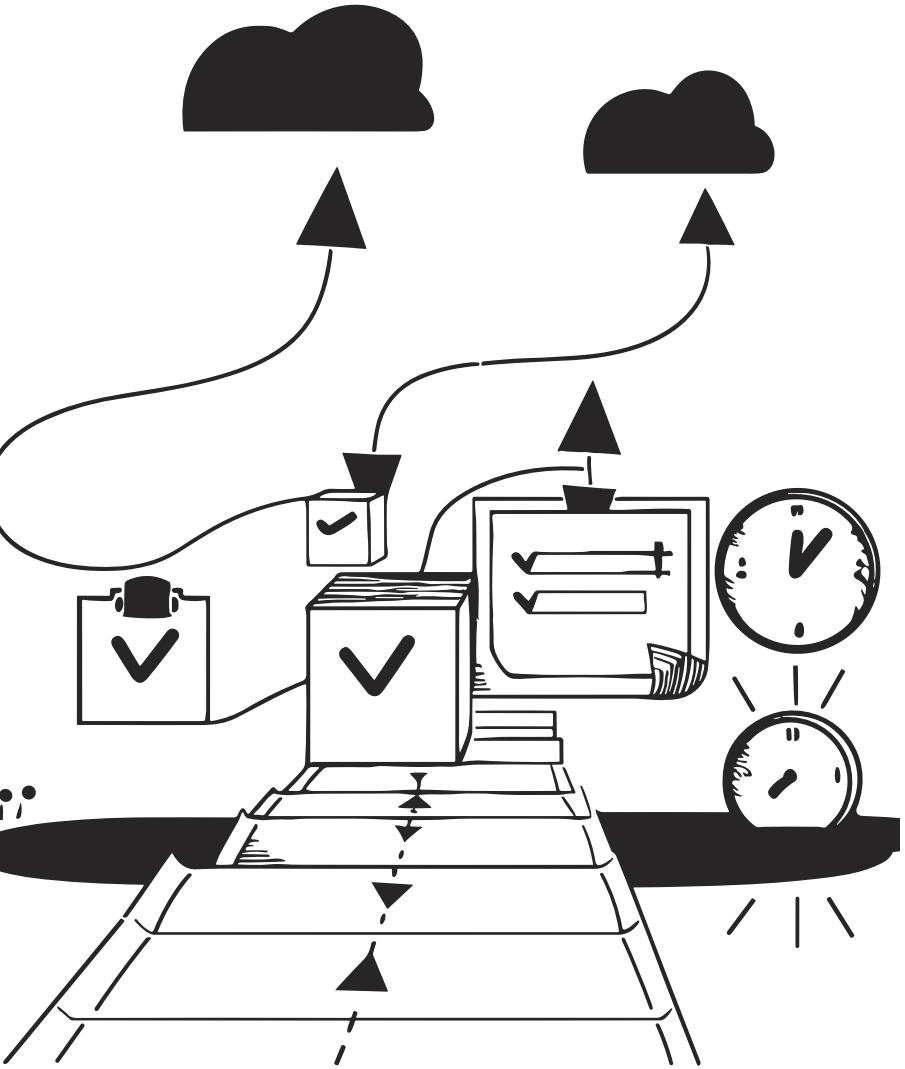
Reactive by design. They find bugs in code that already exists, like SQL injection, XSS, buffer overflows. They cannot tell you that a PR just introduced outbound network access to a domain never in your architecture.

Manual Threat Modeling

Doesn't scale. A 45-minute threat modeling session per feature can't keep pace with teams shipping 50+ PRs per week. And it relies on developers self-reporting what they built — the exact failure mode we're solving.

Code Review

Focuses on the wrong thing. Reviewers focus on logic, readability, and correctness. They're not mapping every PR against an approved capability manifest. Especially when the PR was AI-generated and the reviewer is skimming at 4pm on a Friday.



TASS: The Disagreement Engine

Introducing TASS: Threat Analysis & Security Synthesizer, a flight log for your code. So you can clearly see what AI systems did versus what humans actually intended

The engine auto-scans every pull request and compares it against a declarative capability manifest - an explicit, version-controlled record of what your codebase is allowed to do. When a PR introduces a new capability, it flags it and forces an explicit human decision.

> **This PR introduces outbound HTTPS access to api.stripe.com. This capability is not declared in your manifest. CONFIRM or REVERT.**

Every capability is either explicitly approved or explicitly rejected. The manifest becomes the single source of truth.

What TASS Does For You

TASS helps you catch unapproved architectural decisions before they become permanent.

For Engineering Teams:

Every PR is automatically scanned for new capabilities — external API calls, database writes, network egress, new dependencies. No manual threat modeling. No security questionnaires. Developers just get a clear prompt: this is new, confirm or revert.

For Security Teams:

One declarative manifest per repo becomes your single source of truth for what the codebase is allowed to do. Every capability is explicitly approved or rejected, with a full audit trail. No more "nobody told me."

For Infrastructure/Platform:

TASS auto-generates Kubernetes NetworkPolicies, AWS IAM policies, and security group rules from the capabilities it detects. Your infrastructure policies evolve with your code — automatically.

Our Three-Layer Detection Architecture

What's under the hood?

01

Rapid Dependency Diffing

Catches most of the new capabilities by analyzing changes to package manifests (package.json, go.mod, requirements.txt). New dependency? That's a new capability. **Fast, deterministic, fewer false positives.**

02

Deep Analysis

Tree-sitter-powered static analysis parses actual code changes to detect capabilities that don't appear in dependency files. Think: direct HTTP calls, file system access, database connections, environment variable reads.

03

Policy Generation

Auto-generates infrastructure-as-code policies from detected capabilities. Kubernetes NetworkPolicies from network access patterns. AWS IAM policies scoped to actual resource usage. Security group rules from real egress behavior.

Your infrastructure policies stay in sync with your code. Automatically.

Our Zero Exfiltration Architecture

Your code never leaves your infrastructure.

Single Binary

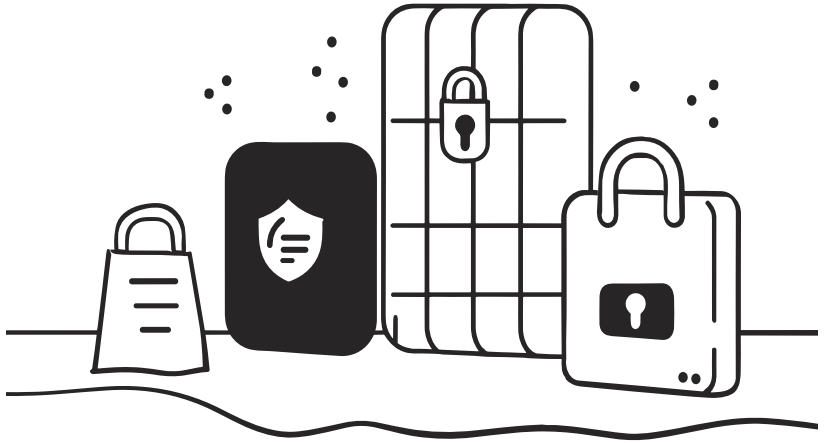
One Go binary. Full engine. No microservice sprawl, no external dependencies at runtime.

Air-Gap Ready

Run TASS entirely within your infrastructure — air-gapped capable. Or use our hosted platform for frictionless GitHub integration. Your choice.

Manifest as Code

The capability manifest lives in your repo as a YAML file. Version-controlled, auditable, reviewable — just like your code.



AI Code Generation Changes the Threat Model

The traditional security model assumed a human wrote every line of code and understood what it did. **That assumption is broken.** AI assistants introduce capabilities developers may not even realize are present. A prompt like "add file upload" might generate code that also sends file metadata to external logging or analytics services without explicitly being asked to.

55%

Faster Task Completion with AI

Security review capacity, on the other hand, hasn't kept up.

96%

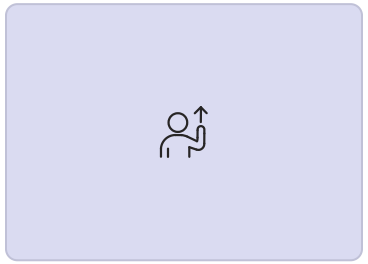
Developers say AI isn't fully correct

Yet **52%** of developers skip reviewing AI code entirely.

The gap between code generation speed and security review capacity is growing exponentially. We close it at the PR level - the last checkpoint before code enters the codebase.

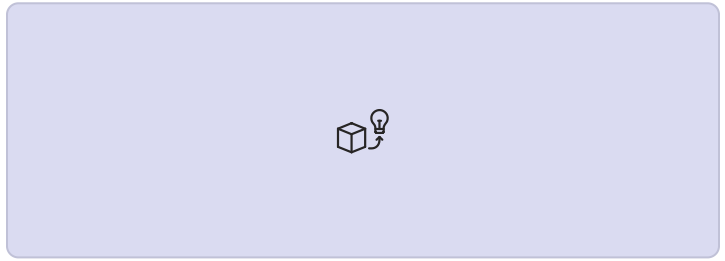
Source: State of Code (Sonar), GitHub Research, Snyk/Stack Overflow

The 1-10-100 Rule: Fix It Early, Save 100x



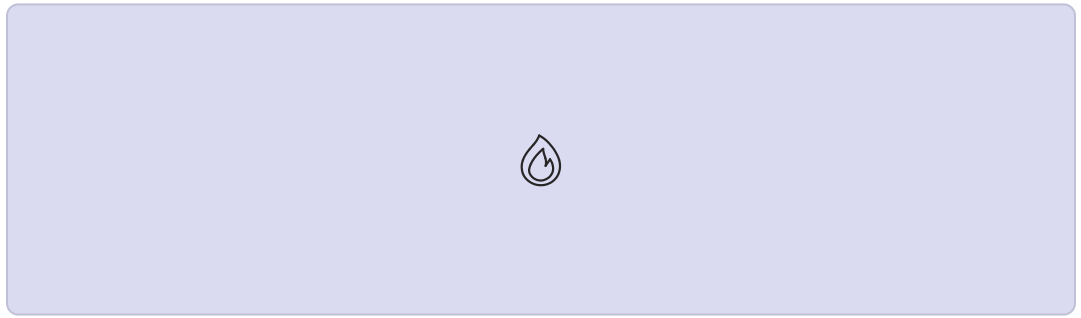
PR Phase

Costs $1. Catch an unapproved capability at the PR. Developer confirms or reverts. Total cost: one conversation.



Development Phase

Costs $10. Capability discovered during integration testing. Requires rework, retesting, schedule impact.



Production

Costs $100. Discovered during incident response. Hotfix, post-mortem, potential breach notification, regulatory exposure.

The average data breach costs **$4.88M (IBM, 2024)**. Architectural flaws found late require fundamental rework and not just patching.

Targeting the AI Code Security Gap

$10.88B

DevSecOps Market

Global Average. Growing at a CAGR of 22.10%, reaching $29.52B by 2030.

45%

Code generated by AI

Introduced security vulnerabilities, across 100+ LLMs

51% → 95%

Developers using AI tools daily

2026 today, projected by 2027

Our sweet spot is **Series A–C scale-ups** in fintech, e-commerce, and defense-adjacent industries — teams shipping fast with AI assistance who need capability governance without slowing down. No dominant player exists yet in AI code governance.

Architectural design flaws in AI-generated code spiked **153%** in 2025, while privilege escalation paths jumped **322%** — exactly the class of flaws traditional scanners miss.

(Source: Apiiro, Grand View Research, Veracode (2025))

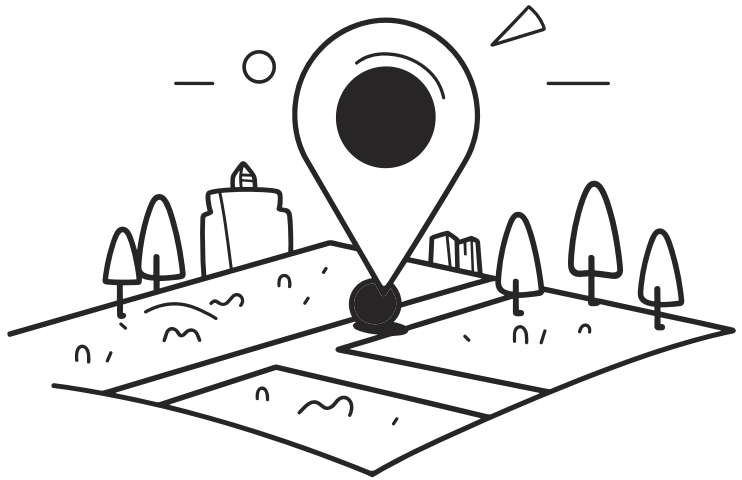
Current Status

What's Built: Core detection engine operational: manifest parsing, dependency diffing, and capability detection are working. GitHub App integration for one-click PR scanning done.

What's In Progress: UI flows, GitHub integration polish, and the overall developer experience layer. The engine runs; now we're making it seamless.

What's Next:

- Auto-generated policy output (K8s NetworkPolicies, IAM)
- Dashboard for capability audit trail and manifest management
- Design partner pilot with real production repos



Join Us as a Design Partner

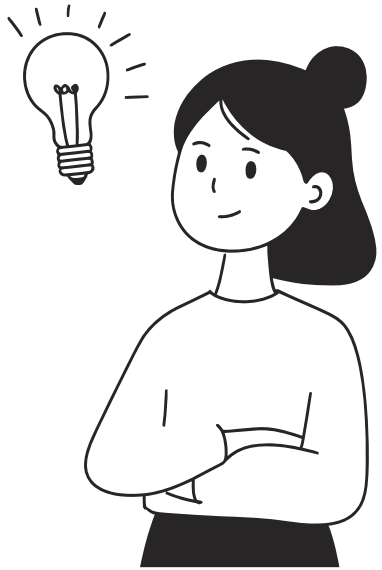
Who We're Looking For

- CTOs or Security Engineers at Series A-C companies
- Teams shipping code with AI coding assistants
- Organizations where unapproved capabilities have caused incidents
- Leaders building in regulated or defense-adjacent environments

What We're Offering

- Free prototype scan on a single microservice
- No funding ask: just a validation partnership
- Proof of value before any commitment
- Direct influence on product roadmap

If unapproved capabilities have ever surprised your team in production, let's talk.



Thank You.

sreenidhin2002@gmail.com | +91 9611642552 | LinkedIn